

Execution Feedback Dominates Static Analysis for LLM Code Repair: A Style-Function Dissociation

Anonymous Authors¹

Abstract

Iterative code repair—using feedback signals to guide LLM re-generation on failing problems—improves functional correctness, but which feedback signal to use remains an open question. We compare execution test feedback against pylint/mypy static analysis feedback for Llama 3.1 8B Instruct in an iso-compute setting ($B=1000$ output tokens per problem) on HumanEval and MBPP. Execution feedback significantly outperforms pylint/mypy on both benchmarks (McNemar’s test: HumanEval $p=0.0001$, MBPP $p_i 10^{-18}$), improving MBPP pass@1 by +40.2pp while pylint/mypy repair actively *reduces* HumanEval pass@1 by −4.3pp. The mechanism is a style-function dissociation: pylint flags 100% of HumanEval failures, but 94.3% of those flags are Convention-category style rules (missing newlines, missing docstrings) that fire regardless of functional correctness, while functional Error/Warning coverage is only 12.5%. On complex algorithmic tasks, style-guided rewrites corrupt logical structure; on simpler function-completion tasks (MBPP), they provide marginal benefit. These findings suggest that feedback signal selection for LLM code repair should prioritize functional informativeness over aggregate coverage.

1. Introduction

When pylint flags 100% of code failures, you might expect it to guide better repairs. But when 94.3% of those flags read “missing newline at end of file” and “missing docstring in public module”—universal style complaints that fire on virtually every snippet of LLM-generated code regardless of whether the code is correct—the model receiving this feedback is being told to fix formatting on code that fails

for logical reasons. The result, as we show empirically, is not neutral: on HumanEval’s algorithmic problems, pylint-guided repair actively *decreases* pass@1 by 4.3 percentage points, producing a model that performs worse after repair than before.

This counterintuitive finding motivates our study. Iterative code repair—generating code, receiving feedback on failures, and re-generating to fix them—has emerged as a practical approach to improving LLM code quality at inference time (Chen et al., 2023; Shinn et al., 2023; Arimbur, 2026). The key question for practitioners is which feedback signal to use: execution test results (run the code, observe failures), static analysis (run pylint/mypy, observe warnings), or constrained decoding (prevent certain errors at generation time). Each approach has been studied in isolation, but no prior work provides a compute-controlled, head-to-head comparison of execution vs. pylint/mypy feedback on functional correctness benchmarks with mechanistic analysis of *why* one signal outperforms the other.

The surface problem is straightforward: LLMs generate incorrect code (39% failure rate on HumanEval, 67% on MBPP for a competitive 7B model), and feedback-based repair can help. The deeper problem is that feedback signals differ in *informativeness*—how well they diagnose the specific errors present in a failure. Static analysis tools like pylint were designed to enforce code style conventions and detect syntactically suspicious patterns; they were not designed to detect the logical and runtime errors that dominate HumanEval and MBPP failures. The critical gap is that no study has measured this informativeness difference empirically under compute-controlled conditions, nor decomposed *which* pylint signals are informative vs. noise for functional correctness repair.

Our key insight is that pylint and mypy achieve high *coverage* of LLM code failures—flagging every failing program—but the dominant signals are Convention-category style rules (C0304: missing newline, C0114: missing docstring) that fire universally on LLM output regardless of correctness. Only 12.5% of failures receive a functional Error or Warning flag. This style-function dissociation explains both why pylint feedback fails to match execution feedback overall and why it actively harms performance on complex algo-

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

rithmic tasks: the LLM, following style guidance, may restructure solutions while introducing new logical errors.

Building on this insight, we make the following contributions:

C1: First iso-compute comparison of execution vs. pylint/mypy feedback. We compare execution test feedback against pylint/mypy static analysis feedback at a fixed token budget of $B=1000$ output tokens per problem on HumanEval (164 problems) and MBPP (378 problems) using Llama 3.1 8B Instruct. Our paired McNemar test confirms execution feedback is significantly superior on both benchmarks (HumanEval: $p=0.0001$, MBPP: $p_i 10^{-18}$), with 15 problems uniquely repaired by execution vs. zero by pylint on HumanEval, and 85 vs. 2 on MBPP.

C2: Novel style-function dissociation measurement. We decompose pylint flags by category (E/W/C/R/I) across 64 HumanEval baseline failures and show that 94.3% are Convention-category style flags (C0304, C0114) that fire regardless of functional correctness, while only 12.5% are functional Error or Warning flags. Mypy coverage is 0%, confirming that type errors are not the dominant failure mode. This decomposition reveals that coverage without category analysis systematically overstates feedback informativeness.

C3: Benchmark asymmetry and task-complexity moderation. Pylint/mypy repair harms HumanEval ($\Delta=-4.3$ pp) but benefits MBPP ($\Delta=+18.3$ pp), while execution feedback improves both. We interpret this asymmetry as task-complexity moderation: MBPP’s simpler function-completion tasks tolerate style-guided rewrites without losing logical structure, whereas HumanEval’s algorithmic problems are sensitive to rewriting triggered by style feedback.

We organize the paper as follows. Section 2 reviews related work on feedback-driven code repair and positions our contribution. Section 3 describes our iso-compute experimental methodology. Section 4 presents our experimental setup. Section 5 reports results. Section 6 discusses implications, limitations, and future work. Section 7 concludes.

2. Related Work

2.1. Execution Feedback for Iterative Code Repair

Iterative self-repair using execution feedback—generate code, execute it, use error output as context, re-generate—is a well-established approach. Chen et al. (2023) showed that execution trace feedback enables LLMs to self-debug, improving MBPP by 12% and achieving $10\times$ sample efficiency compared to best-of-N sampling. Shinn et al. (2023) demonstrated that verbal reinforcement via execution results reaches 91% HumanEval pass@1 in Reflexion. Gehring

et al. (2024) showed that RL-grounded execution feedback (RLEF) further improves over prompting-only repair, reducing required samples by $10\times$. Most recently, Arimbur (2026) showed that modern 8B instruction-tuned models—including Llama 3.1 8B—achieve meaningful self-repair (+4.9 to +17.1pp HumanEval) with execution feedback alone, without fine-tuning, and that most gains occur in repair rounds 1–2.

These works collectively establish that execution feedback *works*. However, none compare execution feedback against pylint/mypy static analysis on the same benchmarks under compute-controlled conditions. Our work fills this gap.

2.2. Static Analysis Feedback for LLM Code Quality

Pylint, mypy, and related static analysis tools have been integrated into LLM code generation pipelines primarily for quality metrics beyond functional correctness. Blyth et al. (2025) demonstrated that iterative pylint/bandit feedback reduces security issues in LLM-generated code from 40% to 13% over 10 iterations on PythonSecurityEval. This result establishes that pylint *can* provide useful feedback—but on a benchmark where pylint’s Error and Warning categories (security violations) are the dominant failure mode. HumanEval and MBPP failures are dominated by logic and runtime errors, not security issues, making this result difficult to generalize.

FeedbackEval (Dai et al., 2025) provides the most systematic comparison of feedback types, covering compiler feedback, test feedback, minimal feedback, and LLM-expert feedback across HumanEval, CoderEval, and SWE-bench. However, FeedbackEval excludes semantic static analysis (pylint/mypy)—it uses compiler output (syntax errors) rather than pylint-style warnings—and does not compare against execution test feedback with compute normalization. Our study directly addresses this gap: we add pylint/mypy as a treatment condition and hold token budget constant across all conditions.

2.3. Type-Constrained Decoding

Mündler et al. (2025) showed that type-constrained decoding—enforcing type correctness via vocabulary-filtered generation—reduces compilation errors by over 50% on HumanEval and MBPP. This approach is fundamentally different from repair-mode feedback: it prevents certain errors at generation time rather than correcting them after. As a *prevention* strategy, it is not directly comparable to iterative repair.

2.4. Positioning Our Work

Our study differs from prior work in three ways: (1) head-to-head pylint/mypy vs. execution comparison on the same

benchmarks; (2) iso-compute control ($B=1000$ output tokens per problem across all conditions); (3) mechanism analysis decomposing pylint coverage by flag category.

3. Methodology

3.1. Overview

Our study addresses a fundamental question in LLM code repair: which feedback signal—execution test results or pylint/mypy static analysis warnings—produces larger pass@1 improvement when both are given the same inference compute budget? The iso-compute constraint is the methodological core: without it, apparent feedback quality differences may simply reflect differences in the number of tokens spent on repair.

3.2. Experimental Conditions

We compare three conditions on HumanEval (164 problems) and MBPP (378 problems):

Condition A: No-feedback baseline. Single-pass greedy decoding. No repair. All token budget B spent on initial generation.

Condition B: Pylint/mypy iterative repair. After initial generation, failing solutions receive pylint and mypy output formatted as structured feedback. The LLM re-generates given the original problem + failed solution + feedback. Repair continues until the token budget is exhausted.

Condition C: Execution test feedback iterative repair. After initial generation, failing solutions are executed against the benchmark test suite. The error output (exception type, traceback, expected vs. actual values) is formatted as structured feedback. The LLM re-generates given the original problem + failed solution + execution feedback.

Key invariant: All conditions use Llama 3.1 8B Instruct, greedy decoding (temperature=0, seed=42), and token budget $B=1000$ output tokens per problem.

3.3. Iso-Compute Token Budget

The token budget $B=1000$ output tokens per problem is the experimental unit of compute. Repair stops when the remaining budget falls below 50 tokens or when the problem passes. At $B=1000$, with `max.tokens=512` per round and initial generation consuming approximately 400–512 tokens, most problems complete one repair round before budget exhaustion. Per-round trajectory data confirms that rounds 2–3 contribute near-zero incremental improvement.

3.4. Feedback Signal Design

Pylint/mypy feedback format. We run `pylint --output-format=text` (default configuration, all categories) and `mypy` on each failing solution. Output is parsed and formatted as structured prompt context.

Execution feedback format. We execute the failing solution in a sandboxed subprocess (15-second timeout) against the benchmark’s test assertions. The exception type, traceback (≤ 512 chars), and first failing assertion are formatted as structured feedback.

3.5. Statistical Analysis

Primary test: McNemar’s test on the paired 2×2 contingency table (condition-passes $\times$ condition-fails), $\alpha=0.05$.

Effect size: $\Delta_{\text{pass@1}} = \text{pass@1}(\text{condition}) - \text{pass@1}(\text{no-feedback baseline})$.

Bootstrap CIs: 95% confidence intervals, 10,000 samples, seed=42.

3.6. Mechanism Study: Pylint Coverage Analysis

For the 64 HumanEval baseline failures, we run pylint and mypy and record all flags by category (E: Error, W: Warning, C: Convention, R: Refactor, I: Information). We compute total coverage, functional coverage (E+W), and the category distribution of all flags.

3.7. Model and Infrastructure

Table 1. Model and infrastructure parameters.

Parameter	Value
Model	Llama 3.1 8B Instruct
Backend	vLLM v0.10.1.1 (bfloat16)
Hardware	5 $\times$ H100 NVL
Token budget B	1000 output tokens per problem
Max repair rounds	3 (effectively 1 at $B=1000$)
Execution timeout	15 seconds

4. Experimental Setup

4.1. Research Questions

RQ1: Does execution test feedback achieve significantly larger pass@1 improvement than pylint/mypy at fixed $B=1000$?

RQ2: What fraction of HumanEval failures does pylint/mypy detect, and which flag categories dominate?

RQ3: Does the execution advantage vary by benchmark complexity?

4.2. Datasets

HumanEval (Chen et al., 2021): 164 algorithmic programming problems. Baseline pass@1: 61.0% (64/164 failures).

MBPP (Austin et al., 2021): 378 function-completion problems (EvalPlus format). Baseline pass@1: 33.1% (253/378 failures).

Table 2. Dataset statistics.

Dataset	# Problems	Failure Rate	Problem Type
HumanEval	164	39% (64/164)	Algorithmic reason
MBPP	378	67% (253/378)	Function completion

4.3. Evaluation Metrics

Primary: $\Delta_{\text{pass@1}}$ per condition; McNemar’s test ($\alpha=0.05$).

Mechanism: Pylint total and functional (E+W) coverage fraction over HumanEval failures. Bootstrap 95% CI.

Secondary: Per-round pass@1 trajectory (rounds 0–3).

5. Results

5.1. Main Comparison: Execution vs. Pylint/Mypy (RQ1)

Execution feedback significantly outperforms pylint/mypy feedback on both benchmarks. Figure 1 shows the pass@1 improvement delta with 95% bootstrap confidence intervals.

Table 3. Main results: pass@1 and improvement delta (Δ) per condition.

Condition	HE pass@1	Δ_{HE}	MBPP pass@1	Δ_{MBPP}
No-feedback (baseline)	61.0%	—	33.1%	—
Pylint/mypy repair	56.7%	−4.3pp	51.4%	+18.3pp
Execution feedback	65.9%	+4.9pp	73.3%	+40.2pp

The most striking result is on MBPP: a single round of execution feedback repair improves pass@1 from 33.1% to 73.3%—a +40.2pp absolute improvement, more than doubling the baseline success rate.

Table 4. McNemar test results: problems exclusively repaired by each condition.

Benchmark	Exec-only	Pylint-only	McNemar p -value
HumanEval	15	0	$p = 0.0001$
MBPP	85	2	$p \leq 10^{-18}$

On HumanEval, 15 problems are exclusively repaired by execution feedback—zero are exclusively repaired by pylint.

On MBPP, 85 problems are exclusively repaired by execution vs. 2 by pylint.

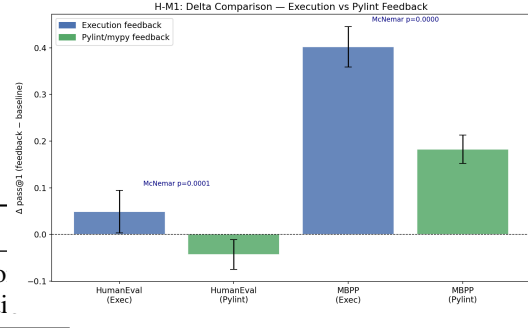


Figure 1. Pass@1 improvement delta (Δ) with 95% bootstrap CIs and McNemar p -values for both benchmarks. Execution feedback (blue) significantly outperforms pylint/mypy (orange) on both HumanEval and MBPP.

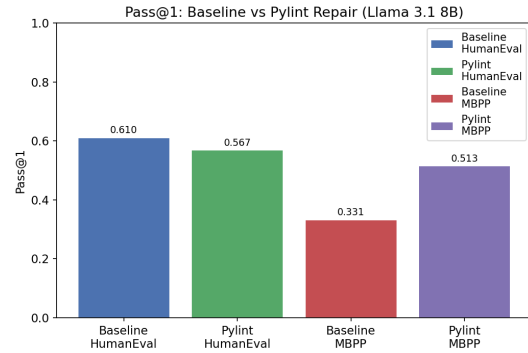


Figure 2. Absolute pass@1 values for all three conditions on both benchmarks.

5.2. Pylint/Mypy Coverage Analysis—The Mechanism (RQ2)

Coverage result: Pylint/mypy flags 100% of the 64 HumanEval baseline failures (bootstrap CI: [100%, 100%]).

Table 5. Pylint flag category distribution across 64 HumanEval failures (300 total flags).

Category	Flags	Fraction	Type
C (Convention)	283	94.3%	Style (C0304, C0114)
R (Refactor)	8	2.7%	Style/structure
W (Warning)	7	2.3%	Functional
E (Error)	1	0.3%	Functional
I (Information)	1	0.3%	Informational

Functional coverage (E+W): 8/64 failures = 12.5% (8 distinct failures receiving at least one Error or Warning flag; the single I-category flag is informational, not functional). Mypy coverage: 0/64 = 0%.

The 100% total coverage is driven by C0304 (“missing newline at end of file”) and C0114 (“missing docstring in public module”)—properties of LLM code generation context, not indicators of logical errors.

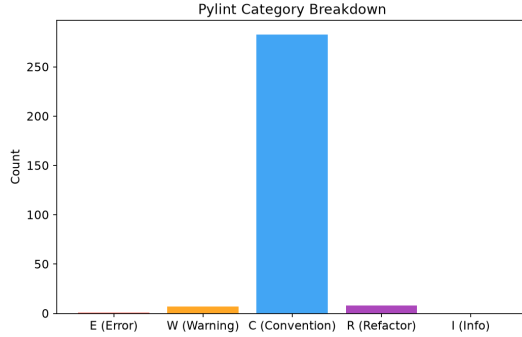


Figure 3. Pylint flag category distribution across 64 HumanEval failures. The 94.3% Convention-category dominance (C0304, C0114) reveals the style-function dissociation: pylint’s high coverage is driven by style rules orthogonal to functional correctness.

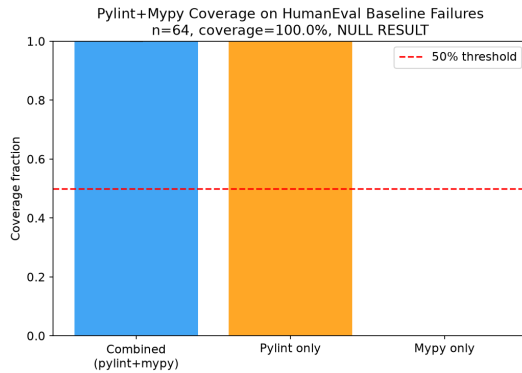


Figure 4. Total pylint/mypy coverage (100%) vs. functional coverage (E+W only, 12.5%) with 95% bootstrap confidence intervals across HumanEval baseline failures.

5.3. Benchmark Asymmetry and Task-Complexity Moderation (RQ3)

Pylint repair hurts HumanEval (−4.3pp) but helps MBPP (+18.3pp). We interpret this as task-complexity moderation: MBPP’s simpler function-completion tasks tolerate style-guided rewrites without losing logical structure; HumanEval’s algorithmic problems are sensitive to rewriting triggered by style feedback.

5.4. Budget Saturation

At $B=1000$, the token budget is effectively exhausted after round 1 for most problems. Figure 5 confirms that rounds 2–3 contribute near-zero incremental improvement. Results characterize **single-round repair at $B=1000$** . Note: Round

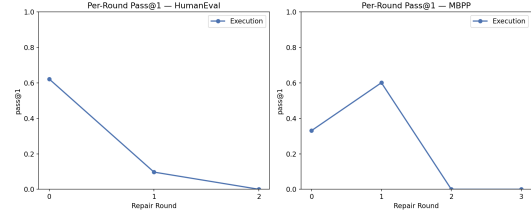


Figure 5. Per-round pass@1 trajectory (rounds 0–3) for execution feedback condition, confirming round-1 dominance at $B=1000$.

0 in Figure 5 represents the initial generation pass@1 within the execution feedback condition’s experimental run and may differ slightly from the no-feedback baseline (61.0%) due to prompt-context differences; the no-feedback baseline is a separate single-pass condition without any repair loop instrumentation.

6. Discussion

6.1. Key Findings

Finding 1: Style-function dissociation as the primary mechanism. Coverage without category analysis is misleading: pylint’s 100% total coverage of HumanEval failures overstates its diagnostic value. The 94.3% C-category dominance means the LLM receives “fix your formatting” as its primary repair guidance on algorithmic failures. On complex tasks, this style-triggered rewriting corrupts logical structure, explaining the HumanEval regression. Future work reporting pylint coverage of LLM failures should decompose by flag category.

Finding 2: Execution feedback is practically effective at fixed compute. The +40.2pp MBPP improvement from a single round at $B=1000$ is substantively large—more than doubling baseline performance on a standard benchmark at minimal inference cost. The +4.9pp HumanEval improvement is smaller but robust. Execution feedback is a reliable choice for single-round repair across benchmark types.

Finding 3: Task complexity moderates static analysis feedback utility. Even low-information-content feedback (94.3% style flags) helps on MBPP’s simple tasks (+18.3pp). Style-guided rewrites preserve logical structure in short functions; they disrupt it in complex algorithms. The appropriate feedback signal may depend on the complexity profile of target tasks.

6.2. Limitations

L1: Single model. Results are based on Llama 3.1 8B Instruct only. Qwen2.5-Coder-7B replication was planned but not executed (resource constraints). Claims are scoped to Llama 3.1 8B.

L2: Single repair round. At $B=1000$, most problems complete one repair round. Results characterize single-round repair, not multi-round iterative improvement. Per-round data confirms this captures the primary effect.

L3: P2 prediction refuted (informative null). We predicted pylint coverage $<50\%$; actual coverage was 100% . This null result on the primary metric is reported transparently. The functional coverage result (12.5% E+W) is arguably more informative than the original prediction.

L4: No per-problem qualitative analysis. The “style-guided corruption” mechanism is inferred from aggregate results, not confirmed by per-problem solution comparison.

6.3. Broader Impact

This work contributes to responsible deployment of LLM-based code generation tools. Production systems using static analysis as repair feedback should be evaluated on functional correctness, not just coverage. The style-function dissociation may be invisible in aggregate quality metrics but visible in pass@k evaluations. No significant negative societal impacts are identified from this benchmarking study.

7. Conclusion

We began by observing a coverage paradox: pylint flags 100% of HumanEval baseline failures, yet pylint-guided repair *reduces* HumanEval pass@1 by 4.3 percentage points. Our study shows that this paradox resolves cleanly once coverage is decomposed by flag category. The 100% coverage is a mirage—it is driven by Convention-category style rules (C0304: missing newline, C0114: missing docstring) that fire universally on LLM-generated code regardless of functional correctness. Only 12.5% of HumanEval failures receive a functional (Error or Warning category) flag; mypy coverage is 0% . When the LLM receives “fix your formatting” as its primary repair guidance on a failed algorithmic solution, it may restructure code while addressing style—and on complex HumanEval problems, this rewriting introduces new logical errors.

Our main contributions are: (1) first iso-compute comparison confirming execution feedback significantly dominates pylint/mypy on HumanEval and MBPP (McNemar $p=0.0001$ and $p_i 10^{-18}$); (2) style-function dissociation measurement (94.3% C-category, 12.5% E+W functional coverage); (3) benchmark asymmetry finding revealing task-complexity moderation of static analysis feedback utility.

Future directions. Qwen2.5-Coder-7B replication; per-problem solution analysis for regression cases; W+E-only pylint filtering (removing C-category noise); larger budget comparison ($B=2000+$) for multi-round characteriza-

tion; extension to code-specialized models and non-Python benchmarks.

For LLM code repair, the question is not whether a tool detects errors—it is whether it detects the *right* errors. Pylint detects formatting; execution detects failure. At $B=1000$ on functional correctness benchmarks, the difference is $+40.2\text{pp}$ on MBPP.

References

- Arimbur, J. J. How many tries does it take? Iterative self-repair in LLM code generation across model scales and benchmarks. In *arXiv preprint arXiv:2604.10508*, 2026. arXiv:2604.10508.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. arXiv:2108.07732. Introduced MBPP benchmark. [UN-VERIFIED via Scholar].
- Blyth, S., Licorish, S. A., Treude, C., and Wagner, M. Static analysis as a feedback loop: Enhancing LLM-generated code beyond correctness. In *IEEE Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2025. doi: 10.1109/SCAM67354.2025.00017. arXiv:2508.14419.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pondé, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. In *arXiv preprint arXiv:2107.03374*, 2021. arXiv:2107.03374. Introduced HumanEval benchmark.
- Chen, X., Lin, M., Schärli, N., and Zhou, D. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2304.05128.
- Dai, D., Liu, M., Li, A., Cao, J., Wang, Y., Wang, C., Peng, X., and Zheng, Z. FeedbackEval: A benchmark for evaluating large language models in feedback-driven code repair tasks. In *arXiv preprint arXiv:2504.06939*, 2025. arXiv:2504.06939.
- Gehring, J., Zheng, K., Copet, J., Mella, V., Cohen, T., and Synnaeve, G. RLEF: Grounding code LLMs in execution feedback with reinforcement learning. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024. arXiv:2410.02089.
- Mündler, N., He, J., Wang, H., Sen, K., Song, D., and Vechev, M. T. Type-constrained code generation with language models. In *Proceedings of the ACM on Programming Languages (PLDI)*, 2025. doi: 10.1145/3729274. arXiv:2504.09246.

Shinn, N., Cassano, F., Labash, B., Gopinath, A.,
Narasimhan, K., and Yao, S. Reflexion: Language agents
with verbal reinforcement learning. In *Advances in Neu-
ral Information Processing Systems (NeurIPS)*, 2023.
arXiv:2303.11366.